ANALIZADOR LÉXICO PASCAL – C

DOCUMENTACIÓN DEL TRABAJO ENTRE LOS INTEGRANTES

Equipo 6

Integrantes:

Sánchez Aguilar Luis Ángel
Álvarez Olivera Irving Damián
Ramírez Manzano Jaret
García Barela Ángel Andrés
López Osorio Haniel
Martinez Cesar Zaid
Soriano Martínez Miguel Ángel

Materia: Compiladores

Profesor: David Martínez Torres

Fecha de entrega: Viernes 25 de septiembre del 2026

UNIVERSIDAD TECNOLÓGICA DE LA MIXTECA

Este proyecto consiste en la implementación de un analizador léxico que procese códigos en lenguaje Pascal, todo creado en Flex. Su propósito es recorrer el código fuente, identificar los lexemas que lo componen y clasificarlos mediante tokens. Asimismo, permite registrar los errores léxicos encontrados.
La entrada del analizador es el código fuente. Como resultado se obtiene una tira de tokens, un registro de errores y una tabla de símbolos. Cada token conserva su tipo, lexema, línea y columna de inicio, lo que permite relacionarlo con su ubicación en el código original.

Asignaciones sobre el analizador hecho con Flex:

#	Tarea del Proyecto	Descripción	Entregable	Responsable
1	**Migración a Flex (.l)**	Traducir expresiones de Python a sintaxis Flex (.l) con estados para comentarios y cadenas.	Archivo .l funcional.	Alvarez Olivera Irving Damian Lopez Osorio Haniel
2	**Diseño Visual de la GUI**	Maquetar paneles de código, tabla de tokens, consola de errores y botones de control.	Vista/Layout de la GUI.	Sanchez Aguilar Luis Angel
3	**Implementación GUI - Flex**	Programar la interfaz gráfica para abrir, compilar y ejecutar el analizador léxico, así como visualizar la tira de tokens, la tabla de símbolos y la tabla de errores.	window.py	Ramirez Manzano Jaret
4	**Diseño de Suite de Pruebas**	Elaborar archivos .pas válidos e inválidos	Banco de pruebas (.pas).	Garcia Barela Angel Andres

		para validar todos los casos léxicos.		
5	**Pruebas Integradas y QA**	Ejecutar pruebas de detección de bugs, fugas de memoria y reporte de fallos.	Informe de bugs y correcciones.	Soriano Martinez Miguel Angel
6	**Documentación y Manual**	Redactar el reporte manual de usuario y guía de compilación.	Documento: Manual de usuario.	Martinez Cesar Zaid

Asignaciones de la semana

Día	Objetivo Principal	Tareas Asignadas del Día	Responsables
Lunes 21	**Organización y estructura base**	Revisar el analizador anterior y comenzar la migración de expresiones regulares a Flex. Diseñar el prototipo visual de la GUI (layout, tablas, paneles). Definir el conjunto de tipos de tokens a manejar.	Alvarez Olivera Irving Damian Sanchez Aguilar Luis Angel Lopez Osorio Haniel
Martes 22	**Desarrollo de componentes**	• Terminar el diseño de la GUI y entregarlo a Jaret. • Implementar tokens, palabras reservadas, números, cadenas y comentarios. • Crear programas `.pas` válidos.	Sanchez Aguilar Luis Angel Alvarez Olivera Irving Damian Garcia Barela Angel Andres
Miércoles 23	**Integración inicial**	• Implementar la ejecución del analizador y visualización de resultados. • Documentar el uso de la interfaz.	Ramirez Manzano Jaret Martinez Cesar Zaid

Jueves 24	**Pruebas y correcciones)**	• Implementar la ejecución del analizador y visualización de resultados. • Ejecutar pruebas completas y reportar errores. • Agregar solución de problemas al manual	Ramirez Manzano Jaret Soriano Martinez Miguel Angel Martinez Cesar Zaid
Viernes 25	**Entrega final**	- corregir los errores reportados e integrar la versión final. - Verificar las correcciones y elaborar el informe de QA. • finalizar el manual de usuario. - Revisar que la GUI y las pruebas cumplan con lo solicitado.	Haniel Lopez Osorio Soriano Martinez Miguel Angel Martinez Cesar Zaid Angel Andres

CONCLUSIÓN

El desarrollo de este analizador léxico e interfaz para Pascal demostró la efectividad de combinar herramientas formales como Flex con un entorno gráfico accesible. La correcta especificación de expresiones regulares permitió procesar de manera eficiente la sintaxis del lenguaje, facilitando tanto el reconocimiento preciso de tokens como el reporte claro de errores. La distribución modular entre los siete integrantes fue clave para integrar la lógica de bajo nivel con la interfaz de usuario, entregando una herramienta funcional, robusta y con una sólida base técnica.